

Equipe: #4

Mauricio Medina Gamba - 1537659

Eugène Vachon - 2149187

Thomas Pajic - 2285919

1. Comportement de déplacement de base - **Thomas**

Fichiers modifiés:

- SDTAIController.cpp
- SDTAIController.h

```
FVector DesiredDirection = FVector::ZeroVector;  
StateMachine->Run(ControlledPawn, DesiredDirection);  
  
Navigation(DesiredDirection, DeltaTime);
```

La machine à états détermine la direction souhaitée de l'agent. Celle-ci est transmise à Navigation(), qui calcule et applique le déplacement à chaque frame.

```
const float CurrentSpeed = FMath::Clamp(Velocity.Size() + Acceleration * DeltaTime, 0.0f, CurrentMaxSpeed);  
  
Velocity = MovementDirection.GetSafeNormal() * CurrentSpeed;  
MovementComponent->Velocity = Velocity;
```

La vitesse augmente progressivement selon l'accélération et le temps écoulé. Elle est limitée par la vitesse maximale, puis appliquée..

```
if (!Velocity.IsNearlyZero())  
{  
    ControlledPawn->SetActorRotation(Velocity.GetSafeNormal().Rotation());  
}
```

L'agent est orienté dans la direction de son vecteur vitesse afin que son orientation corresponde à son déplacement.

2. Détection de murs et comportement d'évitement - **Thomas**

```
const FVector Start = ControlledPawn->GetActorLocation();  
const FVector End = Start + NormalizedDirection * WallDetectionDistance;  
  
FCollisionObjectQueryParams ObjectQueryParams;  
ObjectQueryParams.AddObjectTypesToQuery(ECC_WorldStatic);  
  
FCollisionQueryParams QueryParams;  
QueryParams.AddIgnoredActor(ControlledPawn);  
  
const FCollisionShape CollisionShape = FCollisionShape::MakeSphere(WallSweepRadius);  
  
const bool bHit = GetWorld()->SweepMultiByObjectType(  
    Hits, Start, End, FQuat::Identity, ObjectQueryParams, CollisionShape, QueryParams);
```

La fonction DetectWall() effectue un balayage sphérique devant l'agent pour détecter les murs. La distance et le rayon sont paramétrés par WallDetectionDistance et WallSweepRadius. Les résultats des collisions sont stockés dans Hits.

```
FVector AvoidanceNormal = FVector::ZeroVector;

for (const FHitResult& Hit : Hits)
{
    if (!Hit.bBlockingHit)
    {
        continue;
    }

    FVector Normal = Hit.ImpactNormal;
    Normal.Z = 0.0f;

    if (!Normal.IsNearlyZero())
    {
        AvoidanceNormal += Normal.GetSafeNormal();
    }
}

if (AvoidanceNormal.IsNearlyZero())
{
    return CurrentDirection.GetSafeNormal();
}

AvoidanceNormal.Normalize();
```

Les normales des collisions bloquantes sont combinées pour déterminer une direction représentative de l'obstacle, en ignorant la composante verticale.

```
const float CrossZ = FVector::CrossProduct(CurrentDirection, AvoidanceNormal).Z;
float TurnSign = FMath::Sign(CrossZ);

if (FMath::IsNearlyZero(TurnSign))
{
    TurnSign = 1.0f;
}

const float RotationAmount = TurnSign * WallAvoidanceAngle * DeltaTime;

FVector NewDirection = CurrentDirection.RotateAngleAxis(RotationAmount, FVector::UpVector);
NewDirection.Z = 0.0f;

return NewDirection.GetSafeNormal();
```

Le produit vectoriel permet de déterminer le sens de rotation nécessaire. L'angle appliqué dépend de WallAvoidanceAngle et du temps écoulé.

```

if (DetectWall(MovementDirection, EscapeWallHits))
{
    MovementDirection = ComputeWallAvoidanceDirection(MovementDirection, MovementDirection, EscapeWallHits, DeltaTime);
    MovementDirection = MakeDeathFloorDirectionSafe(MovementDirection, NewDirectDeathFloorHits);
    DeathFloorAvoidanceDirection = MovementDirection;
}

```

Lorsqu'un mur est détecté, la direction de déplacement est modifiée avant le calcul final de la vitesse. L'agent continue de se déplacer et contourne progressivement l'obstacle.

3. Détection d'obstacles et comportement d'évitement
4. Détection et collecte de pickup
- 5. Affichage de feedbacks lors de la collecte de pickup**

Fichiers modifiés:

- STDCollectible.cpp
- Constructeur:

```

ASDTCollectible::ASDTCollectible()
{
    PrimaryActorTick.bCanEverTick = true;

    AudioComponent = CreateDefaultSubobject<UAudioComponent>(TEXT("AudioComponent"));
    AudioComponent->SetupAttachment(RootComponent);
}

```

Nous créons et initialisons l'audio component

- Fonction Collect:

```

void ASDTCollectible::Collect(bool isAgent)
{
    GetWorld()->GetTimerManager().SetTimer(m_CollectCooldownTimer, this, &ASDTCollectible::OnCooldownDone, m_CollectCooldownTimer);

    if (isAgent) {
        if (aiPlaySound) {
            // Sound taken from https://pixabay.com/sound-effects/film-special-effects-item-pickup-3-540193/
            AudioComponent->SetSound(Sound);
            AudioComponent->Play();
        }

        if (aiPlayFx) {
            UNiagaraFunctionLibrary::SpawnSystemAtLocation(GetWorld(), VisualEffect, GetActorLocation());
        }
    }

    GetStaticMeshComponent()->SetVisibility(false);
}

```

Nous avons ajouté un paramètre à la fonction pour savoir si c'est un agent qui a ramassé le collectible. Si c'est le cas, on joue un son et l'animation en fonction de s'ils sont activés.

- STDCollectible.h:

```

UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = AI)
    bool aiPlaySound = false;

UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = AI)
    bool aiPlayFx = false;

UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = AI)
    TObjectPtr<USoundBase> Sound;

UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = AI)
    UNiagaraSystem* VisualEffect;

```

On ajoute les propriétés permettant d'activer ou désactiver le son et les effets. On ajoute aussi une variable pour indiquer le chemin vers le fichier source pour chacun.

- SoftDesignTraining.Build.cs:

```

public class SoftDesignTraining : ModuleRules
{
    public SoftDesignTraining(ReadOnlyTargetRules Target) : base(Target)
    {
        PCHUsage = PCHUsageMode.UseExplicitOrSharedPCHs;
        PublicDependencyModuleNames.AddRange(new string[] { "Core", "CoreUObject", "Engine", "InputCore", "AIModule", "Niagara" });
    }
}

```

On a simplement ajouté Niagara aux dépendances pour créer un effet visuel.

- SoftDesignTrainingCharacter.cpp:

```

else if(ASDTCollectible* collectibleActor = Cast<ASDTCollectible>(OtherActor))
{
    if (!collectibleActor->IsOnCooldown())
    {
        OnCollectPowerUp();
        collectibleActor->Collect(!Cast<ASoftDesignTrainingMainCharacter>(this));
    }
}

```

On déplace l'appel à Collect dans la condition car on ne veut pas jouer le son ou les effets si le collectible n'est pas actif. De plus, on ajuste l'appel pour prendre en compte le nouveau paramètre.

6. Comportement de poursuite - Comportement de fuite

Nous avons implémenté une state machine pour gérer les état de l'agent.:

```

UENUM(BlueprintType)
...
enum class AState : uint8
{
    Patrol,
    Chase,
    Flee,
    Collect
};

```

```

UCLASS(Blueprintable)
0 derived Blueprint classes
class SOFTDESIGNTRAINING_API UStateMachine : public UObject
{
    GENERATED_BODY()

public:
    void Run(APawn* AIPawn, FVector& OutDirection);

private:
    AState CurrentState = AState::Patrol;
    FVector ClosestPickupPosition = FVector::ZeroVector;

    bool IsCharacterClose(ACharacter* TargetCharacter, APawn* AIPawn);
    bool IsCharacterInSight(ACharacter* TargetCharacter, APawn* AIPawn);

    void FindPickup(APawn* AIPawn);
    void Transition(APawn* AIPawn);

    FVector Chase(APawn* AIPawn);
    FVector Flee(APawn* AIPawn);
    FVector Collect(APawn* AIPawn);
    FVector Move(APawn* AIPawn);
};

```

La fonction `void UStateMachine::Transition(APawn* AIPawn)` est responsable du changement d'état. Si l'agent est dans l'état Patrol, voir le player proche fait produit un déclenchement à l'état Chase. Si le player prend le power up, dans n'importe quel état l'agent se trouve, il va passer à l'état de fuite.

Chaque état, dans son exécution, retourne un vecteur direction qui représente la direction désirée dans lequel l'agent doit se déplacer.